Sightline engineering methodology and evidence

9 September 2026 · SIH26171 · RV University · engineering candidate v0.1. **Not submission ready.** The internal delivery target is 11 September. The earlier unresolved SIH2171 preparation record is preserved in `process-preparation-history.md`; the supplied goal and retrieved official catalogue resolved the identity to SIH26171.

Research and problem fit

The ISRO statement requires browser-local computer vision, privacy filtering before outbound context, centralized open-weight reasoning and local execution of returned actions. Its five evaluation weights are visual-context accuracy 25%, sensitive-data detection 20%, redaction precision 20%, client resources 20% and end-to-end latency 15%. `Wiki/problem-statement.md`, the official extraction and `Benchmarks/official-rubric.json` retain the source and measurement boundaries. The displayed catalogue date does not replace the team's internal deadline.

Research separates organizer facts, primary technical literature, institution-reported awards, participant anecdotes and our own observations. The 2022-2025 winner sample establishes recurring presentation patterns, not causal predictors of winning. WebPII/WebRedact, PrivWeb and Available but Invisible establish prior privacy-agent work; novelty is an integration hypothesis requiring comparative testing. Source registers retain URL, retrieval date, author/title when available, hashes, evidence type and limits. Full downloaded sources remain in Raw. Two Google documentation HTML originals containing embedded public-site API configuration stay local; shareable copies remove those strings and disclose the transformation and original hash.

Public social discovery was bounded by access. No private WhatsApp channel, inbox or restricted group was harvested. Missing Instagram/Facebook evidence remains a coverage gap. Search snippets are discovery records, not substitutes for retrieved primary sources. SciSpace/Consensus discovery and primary-paper retrieval are recorded separately.

Guardrails and benchmark design

Mandatory gates use pass/fail/unknown. A failed or unmeasured mandatory gate blocks a readiness claim. Delivering an explicitly labeled engineering candidate does not establish competition readiness. Official weights are preserved without inventing unpublished normalization formulas. Judge persuasion probability remains null: neither a readiness score nor a polished deck is a calibrated probability of winning.

Acceptance metrics define their denominator before measurement: scenario coverage, complete user tasks, recovery attempts, findability tasks, independent narrative ratings, client memory/load/CPU and full-task timing. Component latency is not full-flow latency. The original under-200 ms full-flow guardrail remains failed because actual model steps take seconds. Saturation requires two evaluated iterations with the declared marginal improvement limits and all gates passing; it has not occurred. Human validation and broad dataset results remain absent rather than simulated.

Architecture and technology decisions

Choice	Reason against the problem	Alternative and trade-off
Native JavaScript web harness and extension	Same privacy and action code runs in browser; small framework overhead	React/TypeScript improves larger UI tooling but adds no necessary capability to this small surface

Choice	Reason against the problem	Alternative and trade-off
UltraFace RFB-320, ONNX Runtime Web WASM	Actual packaged local CV, permissive upstream license, CPU-compatible inference	ViT/MediaPipe/WebGPU candidates need comparable accuracy/resource measurements; face-only CV does not solve general PII
Reconstructed semantic scene	Original pixels, DOM, field values and URLs have no outbound schema field	Detected-only masking can retain more utility but risks missed sensitive pixels; requires labeled comparison
Node.js, strict Zod schemas	Shared validation, bounded HTTP API, small operational surface	FastAPI offers broader scientific ecosystem but would duplicate schemas/runtime
Qwen2.5:7B via Ollama	Actual offline-deployable open-weight server reasoning	Smaller model may reduce latency; a VLM may interpret richer protected images but must be evaluated
SQLite metadata-only audit	Local persistence without retaining screen context	Multi-instance hosted service would require different storage/authentication and operational review
Static GitHub Pages website	Existing user-authorized hosting, no server secrets or form database	It cannot host the Node/Ollama backend; local setup is explicit

The current request contains approved control IDs/labels, bounded geometry and opaque-region types. The server uses a text LLM interpreting this layout, not a pixel-reading VLM. The task vocabulary is restricted to three synthetic workflows. These constraints make the first integration inspectable but do not satisfy broad browser-agent utility by themselves.

Privacy and security design

Raw capture and model preprocessing stay in client memory. All unknown text/media are excluded independently of face-detection coverage. This avoids treating a missed face as permission to upload its pixels. The preview exposes residual geometry and allowed labels; structural context can still disclose information, so the design does not claim anonymity or zero leakage.

A plan requires strict schemas, a current revision, an approved target and explicit confirmation. The client rechecks target identity, bounds, disabled/inert state and obstruction. Captures expire after 30 seconds. Arbitrary script, URL navigation, typing and form submission are outside the action schema or require manual handling. Open shadow roots have dedicated invalidation and hit testing. This limits action authority but does not certify prompt-injection resistance.

The API requires an exact allowed origin and timing-safe bearer-token comparison; caps request size, rate and concurrency; and returns bounded errors without stack or provider secrets. Token bootstrap is restricted to loopback same-origin UI. Non-loopback configuration requires an explicit HTTPS public origin and secret. Audit persistence stores counts/timings/action type rather than screen contents. Runtime data, token files and environment secrets are ignored by Git. The Docker recipe exists, but no Docker runtime was available to validate it. No public backend is claimed.

Iterations and verification

1. Corrected problem identity from the official catalogue and replaced generic domain placeholders with the actual privacy-agent scope.
2. Built capture, real ONNX inference, protected-scene construction, Node API, real Ollama planning, local reviewed execution and metadata persistence.
3. Fixed encoded-space bundling and static-root normalization failures. Added server entry and traversal tests.

4. Replaced unconstrained model JSON with a full action-wrapper output schema after the real provider omitted the wrapper.
5. Guarded iframe readiness after a real reload race; added proactive scene expiry.
6. Removed redundant ONNX graph inputs and unused training counters while retaining active weights; browser face inference still worked. This is not a dataset-wide numerical-equivalence result.
7. Audited shadow DOM, stale target meaning, malformed CV outputs and resource cleanup. The 37-test suite passes; details are in `decisions/client-audit.md`.
8. Re-ran actual browser Pending > Review confirmations and observed the correct end-state. Model latency remains a failed requirement.
9. Generated a Stitch design, rendered and compared it, rejected fabricated metrics/model claims, then implemented the selected optical composition with original CSS and local assets.
10. Rendered both slide decks to PDF, inspected contact sheets and corrected cropped imagery and cramped text. The six-slide file preserves the supplied reference's content structure; the separate 15-slide talk has 700 seconds of planned slots, not a measured rehearsal.

Unit tests use explicitly synthetic DOM/provider/runtime doubles. Browser evidence uses the actual local model on the synthetic service desk. Neither proves native-extension behavior or general PII accuracy. Dependencies and model/runtime licenses are recorded; a dependency audit is preserved separately.

Design and accessibility audit

The design uses a large asymmetric headline, optical boundary illustration, real prototype evidence, clear section hierarchy and proposed team responsibilities. Stitch supplied visual ideas; unsupported generated telemetry was discarded. No team portraits, awards, registered ID or contact address were invented. GitHub issues provide a real feedback destination. The local-demo dialog explains the operational prerequisite before linking to localhost.

Semantic landmarks, skip navigation, named controls, visible focus, modal focus return, native keyboard behavior, local fonts and reduced-motion CSS are implemented. The architecture selector updates an announced panel. Desktop and 390px checks found no horizontal overflow on the new website. Six fresh Lighthouse 12.8.2 runs-three mobile and three desktop-scored 100 for performance, accessibility, best practices and SEO. These are local automated measurements, not WCAG 2.1 AA certification, a user study or deployed-backend evidence.

Performance and sustainable engineering

Packaged models/fonts eliminate runtime CDN dependencies. Shared browser modules reduce duplicated behavior, and the website needs no application framework or server. Local CV inference is single-thread WASM; its optimized model is small, but the generic WASM runtime is materially larger and must be included in load budgets. Canvas/tensor cleanup covers failure and disposal paths. Server reasoning currently dominates latency. Smaller-model, worker, cold-load, power and task-cost comparisons remain necessary before claiming optimization or energy savings.

Impact and operating model

A proposed institutional pilot would measure sensitive information excluded, useful context retained, task completion, disclosure comprehension, review burden and cost per completed task. No pilot partner, revenue, savings, user count or avoided incident is established. Deploying on existing devices is a practical option, not proof of sustainability. Support ownership and a release/update process must accompany any real institutional rollout.

Completion audit and remaining work

Requirement	Current evidence	Remaining gap
Correct domain and research	Official source and cited Wiki/Raw	Broader competition/social sample where accessible
Browser-local CV and protected context	Actual WASM/fixture run; strict contract tests	Held-out PII/redaction/utility datasets
Server reasoning and client action	Actual Qwen Pending/Review end-state	Broader workflows, native Chrome/Firefox validation
Guardrails and benchmarks	Defined contracts, 37 tests, browser observations, six Lighthouse runs	Full-task latency target, resources, human metrics, saturation
Presentation	Six-slide candidate, 15-slide talk, POTX and PDFs	Registered team details, timed rehearsal, independent judge review
Website	Implemented design and local browser/Lighthouse checks	Final deployed identity and download verification after push
Demonstration fallback	Actual screenshots	Continuous screen recording and verified playback
Team and impact	Supplied names, proposed roles and pilot metrics	Authentic consented photos, contribution evidence, measured impact

The full goal remains active. `PLAN.md`, `ROAST.md`, `Guardrails/guardrails.json` and `Benchmarks/release-status.json` are the continuing readiness ledger. Do not reinterpret a passing static website or a successful push as completion of the competition entry.